

AVERIS × MONASH HACKATHON 2026 — TEAM DEVSELF

Evermon

Reads a shipping-documentation inbox, compares SI against draft BL by meaning, escalates to a person with a stated reason instead of guessing.

01

The Problem

Clerks manually cross-check SI vs BL

- ▶ Seven fields, by hand: shipper, consignee, notify party, load/discharge port, container count, gross weight.
- ▶ Documents never use the same labels — "Port of Loading" vs "Load Port".
- ▶ A missed mismatch puts the wrong consignee on a negotiable shipping document.

02

Why This Isn't a Wrapper

Rule-based, not prompted

- ▶ Alias-table field alignment (dock/fields.py) and four-reason escalation precedence (dock/review.py) are the product.
- ▶ An LLM given "compare these two documents" gets inconsistent field matching and no auditable policy.
- ▶ Escalation reasons are fixed, never model-improvised — that's the pitch that survives a technical judge's follow-up.

03

Market — Who Pays

Freight forwarders & shipping-line back offices

- ▶ Buyer: the ops manager who owns error/rework cost and demurrage exposure — not the individual clerk.
- ▶ Real B2B wedge, but a crowded category — needs one sharp number, e.g. clerk-hours saved per 100 emails.

04

Open-Source Strategy

Ship the trust layer, keep the moat

- ▶ Open: classification rules, alias table, escalation logic (dock/classify.py, fields.py, review.py) — auditability is the trust argument.
- ▶ Closed: document-parsing edge cases and any fine-tuned extraction — the maintenance moat, not "the loader".

05

Live Demo

Three steps, under two minutes

- ▶ Classify: spam, invoice chase, and a real SI+draft-BL pair — sorted correctly in one pass.
- ▶ Relabelled fields ("Load Port" vs "Port of Loading") still align and flag a real mismatch.
- ▶ A missing/unreadable attachment escalates to NEEDS_REVIEW with the specific reason, not a guess.

Verified against the organisers' own grader

1.0000

overall score — classification,
comparison, escalation

129

deterministic tests — no network, no
LLM

46 / 46

real defect emails caught, 20 / 20
correctly escalated

06

Technical Architecture

Three parts, one shared core

- ▶ dock/: classify -> extract -> compare -> escalate. Stdlib Python 3.11, the graded submission.
- ▶ watch/: Go CLI + terminal dashboard, reuses dock.pipeline.decide() live — not a second implementation.
- ▶ web-demo/: static Cloudflare Worker, a parity-verified TypeScript port of just the comparison rules.

07

Implementation Details

What actually makes it accurate

- ▶ Classification reads the body first, not the subject — this corpus deliberately mislabels it.
- ▶ Field alignment is table-driven: an unrecognised label returns nothing, never a guess.
- ▶ No OCR, no vision model — scanned PDFs escalate correctly instead of inventing a value.

08

Challenges Faced

Two scoring incidents, one operational

- ▶ 91 "please send the BL" emails were first misclassified GENERAL — the scoreboard caught it.
- ▶ That fix alone spiked false escalations 20 -> 111, fixed with a narrower rule.
- ▶ A stray background daemon raced a live correction during testing — an operational gap.

09

Future Roadmap

What's next, and what's deliberately not built yet

- ▶ An audited OCR/vision fallback for scanned PDFs, with a stated confidence and escalation on doubt.
- ▶ Push-based escalation (webhook/notification) instead of a file a person has to poll.
- ▶ A single-instance lock for the watcher daemon; exercising the untested Docker/HTTP loader path.

Try it yourself

Live prototype: evermon-demo.nazrijz336.workers.dev

Demo video: youtu.be/TE72CMXwmZ0

Source: github.com/ponli550/Evermon

Thank you.